

SYNTHETIX · SECURITY REVIEW

Synthetix Deposit Contract.

A security review of Deposit Contract.

CLIENT	PROJECT	AUDITORS	AUDIT WINDOW	FINAL REPORT
Synthetix	Deposit Contract	Guardian	August 28 – October 12, 2025	October 20, 2025

§ 00

Contents.

49 pages — overview, confidence ranking, scope, 38 findings, disclaimer, and about Guardian.

§ 01	Overview	P. 03
§ 02	Confidence Ranking	P. 04
§ 03	Scope & Classifications	P. 05
	Files in Scope	P. 06
§ 04	Findings	P. 07
	Main Review · 27 findings	P. 10
	Remediation Review 1 · 2 findings	P. 37
	Remediation Review 2 · 9 findings	P. 39
§ 05	Disclaimer	P. 48
§ 06	About Guardian	P. 49

§ 01

Overview.

Guardian reviewed Deposit Contract across 3 review rounds. The final report includes 1 High, 6 Medium, 20 Low, 11 Informational findings.

AUDIT FIRM	REPOSITORY
Guardian	Synthetixio/v4-deposit-contract
CLIENT	METHODOLOGY
Synthetix	Smart contract · Manual · Main Review · Remediation Review 1 · Remediation Review 2
PROJECT	MAIN REVIEW COMMIT
Deposit Contract	7211bde467df
PREPARED BY	REMEDATION REVIEW 2 COMMIT
Guardian	01d9c54fefe7
AUDIT WINDOW	
August 28 – October 12, 2025	
FINAL REPORT	
October 20, 2025	

1.1 Confidence Ranking

4 / 5

High Confidence

Guardian assigns a Confidence Ranking of 4 out of 5, combining finding severity, remediation outcome, and the reviewer’s assessment of latent risk. The full scale is on the next page.

1	2	3	4	5
VERY LOW CONFIDENCE	LOW CONFIDENCE	MODERATE CONFIDENCE	HIGH CONFIDENCE	VERY HIGH CONFIDENCE

1.2 Findings at a Glance

SEVERITY	TOTAL	PENDING	ACK	PARTIAL	RESOLVED
Critical	0	0	0	0	0
High	1	0	1	0	0
Medium	6	0	3	0	3
Low	20	0	11	0	9
Informational	11	0	7	0	4
All severities	38	0	22	0	16

§ 02

Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

CONFIDENCE RANKING	DEFINITION AND RECOMMENDATION	RISK PROFILE
5 Very High Confidence	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: code is highly secure at time of audit.	0 High/Critical findings and few Low/Medium severity findings.
4 High Confidence ASSIGNED	Code is clean, well-structured, and adheres to best practices. Design patterns are sound and test coverage is strong. Recommendation: suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity findings.
3 Moderate Confidence	Code is functional but shows gaps in defensive programming or test coverage. Recommendation: remediate the findings and re-review the changes before deployment.	Some High/Critical findings, with various Medium severity findings.
2 Low Confidence	Recurring issues suggest systemic gaps rather than isolated mistakes. Recommendation: remediate, then commission a follow-up review at a frozen commit.	Over 6 High and Critical findings across the engagement.
1 Very Low Confidence	The volume and severity of findings indicate the codebase is not ready for production. Recommendation: substantial rework, then a full re-review.	Over 12 High and Critical findings across the engagement.

§ 03

Scope & Classifications.

Guardian reviewed Deposit Contract in `Synthetixio/v4-deposit-contract`.
The scope includes 763 source lines across 4 files.

3.1 Severity Matrix

	IMP · HIGH	IMP · MED	IMP · LOW
LIK · HIGH	Critical	High	Medium
LIK · MED	High	Medium	Low
LIK · LOW	Medium	Low	Low

IMPACT · HIGH
Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.

IMPACT · MEDIUM
Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.

IMPACT · LOW
Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.

LIKELIHOOD · HIGH
Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.

LIKELIHOOD · MEDIUM
Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.

LIKELIHOOD · LOW
Unlikely to ever occur in production.

Files in Scope.

FILE	NSLOC	TOTAL
src/SyntheticDepositContract.sol	642	1,007
src/libraries/ReasonCodes.sol	17	31
src/interfaces/IPermit2.sol	18	35
src/interfaces/ISyntheticDepositContract.sol	86	293

§ 04

Findings.

Grouped by round. Visible IDs reset by severity within each round, so a finding is named the same way in the report as it was in the channel it was written in.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
MAIN REVIEW					
H-01	Profitable Traders Unfunded	Logical Error	HIGH	ACKNOWLEDGED	p. 10
M-01	Token Pause Not Enforced On Disbursal	Validation	MEDIUM	RESOLVED	p. 11
M-02	Expired Withdrawals Are Disbursed	Logical Error	MEDIUM	RESOLVED	p. 12
M-03	Requested Funds Used As Collateral	Logical Error	MEDIUM	ACKNOWLEDGED	p. 13
M-04	Cancellations When Validated Or Disputed	Logical Error	MEDIUM	ACKNOWLEDGED	p. 14
M-05	Lack Of Deposit Fee	Warning	MEDIUM	ACKNOWLEDGED	p. 15
M-06	Blocked Withdraws For Non-USDT User Balance	Logical Error	MEDIUM	RESOLVED	p. 16
L-01	Balance Validation Can Be Gamed	Validation	LOW	ACKNOWLEDGED	p. 17
L-02	Redundant Dispute Withdrawals	Superfluous Code	LOW	RESOLVED	p. 18
L-03	Inaccurate User Balance Documentation	Documentation	LOW	ACKNOWLEDGED	p. 19
L-04	No Subaccount Validation	Warning	LOW	ACKNOWLEDGED	p. 20
L-05	Blacklisted Tokens Prevent Disbursal	Warning	LOW	ACKNOWLEDGED	p. 21
L-06	Lack Of Validation On Quorum Count	Warning	LOW	RESOLVED	p. 22

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
L-07	Disputed Withdrawals Can Lock Out Users	Warning	LOW	ACKNOWLEDGED	p. 23
L-08	Lack of Minimum Withdrawal Amount	Best Practices	LOW	RESOLVED	p. 24
L-09	Override Status Prevented During Pause	Validation	LOW	RESOLVED	p. 25
L-11	User Max Not Validated With Global Max	Validation	LOW	RESOLVED	p. 26
L-12	Permit2 Not Supported	Validation	LOW	RESOLVED	p. 27
L-13	Batch Dispute Asymmetry	Logical Error	LOW	RESOLVED	p. 28
L-14	Watcher Votes Reuse After Dispute	Logical Error	LOW	ACKNOWLEDGED	p. 29
L-15	Validation Reached Without Watcher Quorum	Unexpected Behavior	LOW	ACKNOWLEDGED	p. 30
L-16	CoW Integration Trust Assumptions	Warning	LOW	RESOLVED	p. 31
L-17	Fee-on-Transfer Tokens Not Supported	Warning	LOW	ACKNOWLEDGED	p. 32
I-01	Superfluous whenNotPaused Modifier	Superfluous Code	INFORMATIONAL	RESOLVED	p. 33
I-02	Finalized States Repeated Throughout Contract	Best Practices	INFORMATIONAL	RESOLVED	p. 34
I-03	Outdated Deposit Contract Technical Spec	Documentation	INFORMATIONAL	ACKNOWLEDGED	p. 35
I-04	Cancel Stale Withdrawals Does Not Mark Reason	Warning	INFORMATIONAL	RESOLVED	p. 36

REMEDIATION REVIEW 1

L-01	Users Can Be Trapped Upon Deposit	Validation	LOW	RESOLVED	p. 37
I-01	Disburse Withdrawal NatSpec	Documentation	INFORMATIONAL	ACKNOWLEDGED	p. 38

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
REMEDIATION REVIEW 2					
L-01	Collateral Cannot Be Added In Deploy	Logical Error	LOW	ACKNOWLEDGED	p. 39
L-02	CowSwap Warnings	Warning	LOW	ACKNOWLEDGED	p. 40
L-03	Token Withdrawal Paused After User Requested Withdraw	Warning	LOW	ACKNOWLEDGED	p. 41
I-01	Invalid MockToken Decimals Used For Testing	Logical Error	INFORMATIONAL	ACKNOWLEDGED	p. 42
I-02	Incorrect MIN_ENVELOP_LENGTH Comment	Documentation	INFORMATIONAL	RESOLVED	p. 43
I-03	Request ID Can Be Reused	Warning	INFORMATIONAL	ACKNOWLEDGED	p. 44
I-04	Parameters Can Be More Granular	Warning	INFORMATIONAL	ACKNOWLEDGED	p. 45
I-05	Specific Swap Pause Can Be Added	Warning	INFORMATIONAL	ACKNOWLEDGED	p. 46
I-06	Unclear Error InvalidTradeParams	Warning	INFORMATIONAL	ACKNOWLEDGED	p. 47

HIGH **H-01**

ACKNOWLEDGED

Profitable Traders Unfunded

CATEGORY	Logical Error
LOCATION	SyntheticDepositContract.sol
REVIEW	Main Review

— DESCRIPTION

The Deposit contract never posts on-chain PnL between counterparties. A winning trader's withdrawal can exceed their on-chain user balance and is still disbursed as long as the contract holds enough tokens, while no corresponding debit is applied to the losing trader. This is not zero-sum and allows profits to be realized without charging the other side of the trade, eventually leading to contract insolvency without appropriate risk engine/off-chain withdrawal validations.

Furthermore, the losing trader's user balance is closer to the `config.userMaximum`, so if they would like to deposit more collateral to save their position from being liquidated, they may be prevented from doing so, although their true, off-chain balance is less than the user balance the on-chain mapping contains due to negative PnL.

— RECOMMENDATION

Have the relayer post an order settlement to the contract, such that the user balance of both the maker and taker are adjusted by the PnL.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-01**

RESOLVED

Token Pause Not Enforced On Disbursal

CATEGORY Validation

LOCATION <https://github.com/Synthetixio/v4-deposit-contract/blob/7211bde467df9d715feb13ab567ea65d9ee1081a/src/SynthetixDepositContract.sol#L614>

REVIEW Main Review

DESCRIPTION

Mapping `tokenWithdrawalsPaused` is intended to prevent withdrawals for specific tokens. Function `requestWithdrawal` checks `tokenWithdrawalsPaused[token]` but function `disburseWithdrawal` does not. Consequently assets that are intended to be paused may still leave the Deposit contract.

RECOMMENDATION

Consider validation again `tokenWithdrawalsPaused` in function `disburseWithdrawal`.

RESOLUTION

Status: Resolved.

<https://github.com/Synthetixio/v4-deposit-contract/commit/a99710d2f2fdb1937c7a6160b18b1cebcd7aba70>

MEDIUM **M-02**

RESOLVED

Expired Withdrawals Are Disbursed

CATEGORY	Logical Error
LOCATION	https://github.com/Synthetixio/v4-deposit-contract/blob/7211bde467df9d715feb13ab567ea65d9ee1081a/src/SynthetixDepositContract.sol#L614
REVIEW	Main Review

DESCRIPTION

The contract implements an expiry mechanism for withdrawal requests using function `_isWithdrawalExpired()`, which returns true if a request is in `Requested` or `Validated` states and its withdrawal timeout has elapsed. However, `disburseWithdrawal()` does not check expiry. A teller can disburse a withdrawal that has been in the `Validated` state past the `withdrawalExpiryTimeout`. As a result, withdrawals that are technically expired can still be executed and paid out.

RECOMMENDATION

Consider if expired withdrawals should be disbursable. If not, attempt to cancel the request when disbursing.

RESOLUTION

Status: Resolved.

<https://github.com/Synthetixio/v4-deposit-contract/commit/6a1fd07e4e25f839ad82ae4b281f8ced80392bab>

MEDIUM **M-03**

ACKNOWLEDGED

Requested Funds Used As Collateral

CATEGORY Logical Error
LOCATION SynthetixDepositContract.sol
REVIEW Main Review

DESCRIPTION

When a withdrawal is requested/validated, the contract does not “lock” or put aside those funds. The user’s on-chain balance isn’t reserved since only at disbursal is `userBalance` decreased. Meanwhile, the off-chain component can still potentially treat the same balance as available margin which enables double-use of the same collateral (pending withdrawal and trading margin). This may lead to some scenarios where if a trader’s position quickly turns into a loss, the Teller may disburse more funds than should be allocated to the user based on their latest position.

RECOMMENDATION

Consider setting aside funds that have been requested for withdraw from being used as margin.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-04**

ACKNOWLEDGED

Cancellations When Validated Or Disputed

CATEGORY	Logical Error
LOCATION	https://github.com/Synthetixio/v4-deposit-contract/blob/7211bde467df9d715feb13ab567ea65d9ee1081a/src/SynthetixDepositContract.sol#L486
REVIEW	Main Review

— DESCRIPTION

According to technical documentation, "A user can call `cancelWithdrawal(uint256 _id)` to cancel their own request, but only if it is still in the Requested, Approved, or Disputed states."

Function `cancelWithdrawal` explicitly validates against `Validated` and `Disputed` withdrawal requests and reverts with `InvalidStateForAction` if so. Consequently, user cancellation will be prevented in these states, going against technical spec.

— RECOMMENDATION

Either update the technical spec to reflect the latest contract behavior, or allow for cancellation in these states. Note that race conditions with off-chain component will have to be appropriately handled if cancellations are supported when the Teller can already disburse funds (in Validated state) for example.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-05**

ACKNOWLEDGED

Lack Of Deposit Fee

CATEGORY Warning
LOCATION SynthetixDepositContract.sol: 361
REVIEW Main Review

DESCRIPTION

The Deposit contract lacks a fee on deposits. Consequently, a malicious user may take the entire deposit capacity (the `globalMaximum`) and prevent other users from depositing collateral. Depending on the size of the `userMaximum`, the attacker can simply split up deposits among multiple addresses.

RECOMMENDATION

Consider adding a configurable deposit fee.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-06**

RESOLVED

Blocked Withdraws For Non-USDT User Balance

CATEGORY Logical Error
LOCATION SynthetixDepositContract.sol
REVIEW Main Review

DESCRIPTION

The documentation stats that PnL is settled in USDT, so the contract must keep healthy USDT liquidity. However, the Deposit contract's accounting treats `totalDeposited[token]` as a uint that only increases on user deposits and decreases on disbursement. Trading PnL or CoW conversions that add USDT to the vault do not increase `totalDeposited[USDT]`.

For non-USDT collateral depositors, their PnL is denominated in USDT, but no matching USDT was ever credited to `totalDeposited[USDT]`. When such a user requests a USDT withdrawal, the system subtracts from `totalDeposited[USDT]` even though that counter was never incremented. This can drive `totalDeposited[USDT]` below zero, trigger an underflow revert and blocking withdrawals.

RECOMMENDATION

Calculate the user's USDT PnL off-chain, price that amount into the collateral token deposited by the user (accounting for any valuation haircut as necessary), and relay the withdrawal for that amount. If USDT disbursal is needed, support a withdrawal for that amount.

RESOLUTION

Status: Resolved.

LOW **L-01**

ACKNOWLEDGED

Balance Validation Can Be Gamed

CATEGORY Validation
LOCATION SynthetixDepositContract: 361
REVIEW Main Review

DESCRIPTION

Function `requestWithdrawal` supports multiple withdrawal entries and validates that each entry does not exceed the contract balance.

If there are multiple entries with the same token, the amount that is request for that token can exceed the actual contract balance.

For example:

- (1) Contract balance = 10 WETH
- (2) Two withdrawal entries of 10 WETH
- (3) Each entry passes the validation but contract does not have enough balance to support the total withdrawal of 20 WETH.

Furthermore, the validation does not take into account tokens that have already been requested for withdrawal for a different beneficiary. This may lead to potential issues with on-chain solvency.

RECOMMENDATION

Consider adding more validation on-chain to account for requested tokens from other user withdrawal requests and/or add more off-chain validation such that each token only has one entry.

RESOLUTION

Status: Acknowledged.

LOW **L-02**

RESOLVED

Redundant Dispute Withdrawals

CATEGORY	Superfluous Code
LOCATION	SynthetixDepositContract.sol
REVIEW	Main Review

DESCRIPTION

There are multiple functions for >1 actor to dispute withdrawals:

- (1) `batchDisputeWithdrawals` by RELAYER_ROLE
- (2) `watcherDisputeWithdrawal` by WATCHER_ROLE
- (3) `batchWatcherDisputeWithdrawals` by WATCHER_ROLE
- (4) `disputeWithdrawal` callable by RELAYER_ROLE and WATCHER_ROLE

All of this functionality can be combined into a single function.

RECOMMENDATION

Consider consolidating disputes into a single function such as `disputeWithdrawal` callable by both roles.

RESOLUTION

Status: Resolved.

<https://github.com/Synthetixio/v4-deposit-contract/commit/a57893d86db70ec6ac624cfaddad4ec72bf29d6f>

LOW **L-03**

ACKNOWLEDGED

Inaccurate User Balance Documentation

CATEGORY Documentation
LOCATION ISyntheticDepositContract: 315
REVIEW Main Review

DESCRIPTION

According to documentation, `_userBalance` represents "[t]he user's deposited amount (can be negative if losses exceed deposits)". However, according to the tests within `SuccessfulTraderWithdrawalTest`, trading profit is what leads to the user balance being negative upon withdrawal disbursement. Furthermore, if a user withdraws profit so that their user balance is -X and then deposits X more tokens, their user balance appears as 0 which may cause issues with off-chain infrastructure and may bypass the `config.userMaximum` validation.

RECOMMENDATION

Adjust the documentation to reflect trader profit leading to negative balance, or adjust the implementation so profit increases user balance.

RESOLUTION

Status: Acknowledged.

LOW **L-04**

ACKNOWLEDGED

No Subaccount Validation

CATEGORY	Warning
LOCATION	SynthetixDepositContract.sol
REVIEW	Main Review

— DESCRIPTION

There is currently no subaccount validation on the Deposit contract. For example, a user may end up depositing to a `subAccountId` that does not exist for their address. An `AssetDeposited` event will be emitted with the non-existent subaccount, and it is crucial that the off-chain component appropriately handles this case.

— RECOMMENDATION

Ensure the off-chain component handles currently non-existent subaccounts.

— RESOLUTION

Status: Acknowledged.

LOW **L-05**

ACKNOWLEDGED

Blacklisted Tokens Prevent Disbursal

CATEGORY	Warning
LOCATION	SynthetixDepositContract.sol
REVIEW	Main Review

DESCRIPTION

A Teller will attempt to disburse funds through function `disburseWithdrawal` after withdrawal validation. However, if the token being transferred has blacklist functionality such as USDT and the user has been blacklisted, this will prevent the disbursal until the withdrawal turns stale and is rejected. In the meantime while the withdrawal is not yet expired, it is important that the Teller is appropriately configured so it is not prevented from disbursing other withdrawal requests.

RECOMMENDATION

Ensure the Teller is capable of moving on to other withdrawal requests if a particular request is reverting. Furthermore, validate that the amount being transferred is greater than 0 within both `disburseWithdrawal` and `batchDisburseWithdrawals`.

RESOLUTION

Status: Acknowledged.

LOW **L-06**

RESOLVED

Lack Of Validation On Quorum Count

CATEGORY	Warning
LOCATION	https://github.com/Synthetixio/v4-deposit-contract/blob/7211bde467df9d715feb13ab567ea65d9ee1081a/src/SynthetixDepositContract.sol#L799
REVIEW	Main Review

DESCRIPTION

In order for a request to become validated, the `watcherQuorum` number of votes must be reached and this parameter is configured within function `setWatcherQuorum`. The `watcherQuorum` can be set to a value larger than the number of watches and hence the Requested -> Validated state transition will not be possible.

RECOMMENDATION

Ensure the quorum amount is non-zero but no greater than the number of watchers.

RESOLUTION

Status: Resolved.

<https://github.com/Synthetixio/v4-deposit-contract/commit/ebd58d99f778310b0c459d5fa4e6775672afce7e>

LOW **L-07**

ACKNOWLEDGED

Disputed Withdrawals Can Lock Out Users

CATEGORY Warning
LOCATION SynthetixDepositContract.sol
REVIEW Main Review

— DESCRIPTION

Function `_isWithdrawalExpired()` does not treat Disputed requests as expirable, and users cannot cancel from Disputed. Only a Guardian (or owner override) can resolve them. If Guardians (multi-sigs) are inactive, the request is stuck, and since each user may only have one active withdrawal at a time, the affected user is blocked from creating new withdrawals.

— RECOMMENDATION

Clearly document this risk and ensure availability of Guardians.

— RESOLUTION

Status: Acknowledged.

LOW **L-08**

RESOLVED

Lack of Minimum Withdrawal Amount

CATEGORY	Best Practices
LOCATION	SynthetixDepositContract.sol
REVIEW	Main Review

DESCRIPTION

The contract enforces a minimum deposit amount with `CollateralConfig.userMinimum`, but no equivalent check exists for withdrawals. A relayer can therefore create arbitrarily small withdrawal requests based on off-chain user inputs, which can lead to spam and unnecessary gas wastage.

RECOMMENDATION

Consider adding the ability to set a minimum withdrawal amount.

RESOLUTION

Status: Resolved.

<https://github.com/Synthetixio/v4-deposit-contract/pull/20/commits/8f1ac7e925374bd6df8004b375e2c1a209abc2eb>

LOW **L-09**

RESOLVED

Override Status Prevented During Pause

CATEGORY	Validation
LOCATION	https://github.com/Synthetixio/v4-deposit-contract/blob/7211bde467df9d715feb13ab567ea65d9ee1081a/src/SynthetixDepositContract.sol#L895
REVIEW	Main Review

DESCRIPTION

The `overrideRequestStatus` function is intended to be used as an "[e]mergency function", but is gated by the `whenNotPaused` modifier. Consequently, if the contract is paused during an emergency, the Owner will be unable to override the status.

RECOMMENDATION

Consider removing the `whenNotPaused` modifier from function `overrideRequestStatus`.

RESOLUTION

Status: Resolved.

<https://github.com/Synthetixio/v4-deposit-contract/commit/75499b41fa57ad96dbdd9997fb7cf59afaca1475>

LOW **L-11**

RESOLVED

User Max Not Validated With Global Max

CATEGORY Validation
LOCATION SynthetixDepositContract.sol
REVIEW Main Review

DESCRIPTION

The contract validates `userMaximum` and `globalMaximum` independently when adding or updating collateral configs. However, it does not enforce that `userMaximum <= globalMaximum`. This allows a misconfigured token where `userMaximum` exceeds the global cap, preventing user deposits.

RECOMMENDATION

Consider validating that `userMaximum <= globalMaximum` when setting configs.

RESOLUTION

Status: Resolved.

<https://github.com/Synthetixio/v4-deposit-contract/commit/b1d41a6092b15a23d2f86f89597c4051d8d9c9b2>

LOW **L-12**

RESOLVED

Permit2 Not Supported

CATEGORY	Validation
LOCATION	SynthetixDepositContract.sol
REVIEW	Main Review

DESCRIPTION

The PERMIT2 contract has the following PermitTransferFrom:

```
struct PermitTransferFrom {
    TokenPermissions permitted;
    // a unique value for every token owner's signature to prevent signature replays
    uint256 nonce;
    // deadline on the permit signature
    uint256 deadline;
}
```

which differs from SNX's struct:

```
struct PermitTransferFrom {
    TokenPermissions permitted;
    address spender;
    uint256 nonce;
    uint256 deadline;
}
```

Hence the `spender` is not necessary and will trigger failure in PERMIT2 functionality when a signature is provided to `permitTransferFrom`.

RECOMMENDATION

Update the PermitTransferFrom struct to the PERMIT2 version.

RESOLUTION

Status: Resolved.

LOW **L-13**

RESOLVED

Batch Dispute Asymmetry

CATEGORY	Logical Error
LOCATION	https://github.com/Synthetixio/v4-deposit-contract/blob/7211bde467df9d715feb13ab567ea65d9ee1081a/src/SynthetixDepositContract.sol#L524
REVIEW	Main Review

DESCRIPTION

Function `batchDisputeWithdrawals` does not finalize expired withdrawal requests with `Status.Expires`, which is asymmetrical with `batchRejectWithdrawals` which does first check if a request is expired and setting its status as expired if so.

RECOMMENDATION

Add a check within `batchDisputeWithdrawals` to check whether the withdrawal is expired and set it as expired if so.

RESOLUTION

Status: Resolved.

LOW **L-14**

ACKNOWLEDGED

Watcher Votes Reuse After Dispute

CATEGORY Logical Error
LOCATION SynthetixDepositContract.sol
REVIEW Main Review

DESCRIPTION

Watcher votes are not cleared when a withdrawal request is disputed. If a request in Requested accumulates watcher votes and is then moved to Disputed, those prior votes remain stored. Should the request later return to Requested (through `overrideRequestStatus` for example), the old votes still apply, allowing the request to be quickly re-validated without a fresh round of voting. This potentially undermines the intended integrity of the dispute process.

RECOMMENDATION

Consider resetting watcher votes and `watcherCount` if the request moves away from a Requested state.

RESOLUTION

Status: Acknowledged.

LOW **L-15**

ACKNOWLEDGED

Validation Reached Without Watcher Quorum

CATEGORY	Unexpected Behavior
LOCATION	https://github.com/Synthetixio/v4-deposit-contract/blob/b8cc8fd6c158534d309d81c93e1696015a7cc12a/src/SynthetixDepositContract.sol#L731
REVIEW	Main Review

— DESCRIPTION

A withdrawal request can reach the Validated state without any watcher votes. Specifically, if a request in `Status.Requested` is disputed, a Guardian can call `resolveDisputedWithdrawal(_approve = true)`, which directly transitions the request to `Status.Validated`. This bypasses the watcher quorum mechanism and allows withdrawals to be validated solely through Guardian approval, which may be unexpected behavior since spec states "[a] configurable quorum (M-of-N) of Watchers must approve a request before it can be processed."

— RECOMMENDATION

Clearly document this behavior if intended under the GUARDIAN_ROLE section, clarifying that resolved requests can be processed for disbursal. Otherwise, transition the state to `Status.Requested` and let the watchers then cast votes.

— RESOLUTION

Status: Acknowledged.

LOW **L-16**

RESOLVED

CoW Integration Trust Assumptions

CATEGORY	Warning
LOCATION	https://github.com/Synthetixio/v4-deposit-contract/blob/b8cc8fd6c158534d309d81c93e1696015a7cc12a/src/SynthetixDepositContract.sol#L988
REVIEW	Main Review

— DESCRIPTION

Currently there is no validation on the CoW order within function `isValidSignature` beyond validation that the signer is an authorized trader, leading to serious trust assumptions for authorized traders.

Orders can be submitted that have an unexpected output token, where the receiver is not the Deposit contract nor an authorized receiver leading to drainage, etc. The documentation for SLP Collateral Exchange has example code including validations: https://www.notion.so/OFFICIAL-SLP-Collateral-Exchange-System-2390a80a459e80c2b0f5e166199a14fe?source=copy_link#24c0a80a459e80b8a402f2daf2692ac6

— RECOMMENDATION

Consider adding more validations within function `isValidSignature`. Otherwise, clearly document trust assumptions.

— RESOLUTION

Status: Resolved.

LOW **L-17**

ACKNOWLEDGED

Fee-on-Transfer Tokens Not Supported

CATEGORY	Warning
LOCATION	https://github.com/Synthetixio/v4-deposit-contract/blob/7211bde467df9d715feb13ab567ea65d9ee1081a/src/SynthetixDepositContract.sol#L387
REVIEW	Main Review

— DESCRIPTION

User balance and `totalDeposited` is incremented by the `item.amount`, but this may differ from the actual amount received by the Deposit contract if a fee-on-transfer token was used as collateral.

— RECOMMENDATION

Carefully select which tokens can be used as collateral, avoiding fee-on-transfer tokens, ERC777, etc.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-01**

RESOLVED

Superfluous whenNotPaused Modifier

CATEGORY Superfluous Code
LOCATION SynthetixDepositContract.sol
REVIEW Main Review

DESCRIPTION

Function `deposit` uses modifier `whenNotPaused` which reverts `if (depositsGloballyPaused && withdrawalsGloballyPaused)`

Within the `deposit` function body, `depositsGloballyPaused` is specifically validated with `if (depositsGloballyPaused) revert DepositsGloballyPaused();`

Consequently, the `whenNotPaused` modifier is not necessary since it would only be triggered if `depositsGloballyPaused = true` anyway. The same modifier is also unnecessary on function `requestWithdrawal` since `if (withdrawalsGloballyPaused)` is validated within the function body.

RECOMMENDATION

Consider removing `whenNotPaused` from functions `deposit` and `requestWithdraw`.

RESOLUTION

Status: Resolved.

<https://github.com/Synthetixio/v4-deposit-contract/commit/f91e8fddd12932cff063544770c833b463cd755f>

INFORMATIONAL **I-02**

RESOLVED

Finalized States Repeated Throughout Contract

CATEGORY	Best Practices
LOCATION	SynthetixDepositContract.sol
REVIEW	Main Review

DESCRIPTION

The boolean expression for finalized states - `req.status == Status.Disbursed || req.status == Status.Denied || req.status == Status.Cancelled || req.status == Status.Expired` - is repeated throughout the contract. Consider consolidating the validation in a single function to prevent inconsistencies if future updates/upgrades occur.

RECOMMENDATION

Consider consolidating finalized states into one function.

RESOLUTION

Status: Resolved.

<https://github.com/Synthetixio/v4-deposit-contract/commit/32d81be8a299ad0a65821508cab0347ab35f16b4>

INFORMATIONAL I-03

ACKNOWLEDGED

Outdated Deposit Contract Technical Spec

CATEGORY Documentation
LOCATION Global
REVIEW Main Review

— DESCRIPTION

The existing Deposit contract technical doc has inconsistencies with the latest code, including but not limited to:

- (1) Function `approveWithdrawal` does not exist although in doc it is stated that "the Relayer calls this function to transition the request's status from Requested to Approved."
- (2) `validateWithdrawal` function stated in technical doc does not exist and watchers cast votes with `castWatcherVote` instead.
- (3) The spec mentions event `WithdrawalTimeoutSet` but the Deposit contract uses event `WithdrawalExpiryTimeoutSet`

— RECOMMENDATION

Update the spec to match the latest Deposit contract.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-04**

RESOLVED

Cancel Stale Withdrawals Does Not Mark Reason

CATEGORY	Warning
LOCATION	https://github.com/Synthetixio/v4-deposit-contract/blob/7211bde467df9d715feb13ab567ea65d9ee1081a/src/SynthetixDepositContract.sol#L677
REVIEW	Main Review

DESCRIPTION

Function `cancelStaleWithdrawals` finalizes the request with reason code `ReasonCodes.REASON_NONE`, while other cancellations on expired withdrawal requests through the Deposit contract use `REASON_EXPIRED`.

RECOMMENDATION

Consider if `cancelStaleWithdrawals` should set the reason code as `REASON_EXPIRED`.

RESOLUTION

Status: Resolved.

LOW **L-01**

RESOLVED

Users Can Be Trapped Upon Deposit

CATEGORY Validation

LOCATION <https://github.com/Synthetixio/v4-deposit-contract/blob/8f1ac7e925374bd6df8004b375e2c1a209abc2eb/src/SynthetixDepositContract.sol#L748>

REVIEW Remediation Review 1

DESCRIPTION

Function `updateCollateralConfig` rejects `withdrawalMinimum > **userMaximum**`, but `addCollateral` rejects `withdrawalMinimum > **userMinimum**`, so after updating the collateral's config the withdrawal floor can be raised above the deposit floor and users may be trapped.

RECOMMENDATION

Use the same `if (_config.withdrawalMinimum > _config.userMinimum) revert InvalidInput();` validation in function `updateCollateralConfig`.

RESOLUTION

Status: Resolved.

INFORMATIONAL **I-01**

ACKNOWLEDGED

Disburse Withdrawal NatSpec

CATEGORY	Documentation
LOCATION	https://github.com/Synthetixio/v4-deposit-contract/blob/8f1ac7e925374bd6df8004b375e2c1a209abc2eb/src/SynthetixDepositContract.sol#L585
REVIEW	Remediation Review 1

DESCRIPTION

Function `disburseWithdrawal` implies a payout/disbursal upon successful execution, but execution can exit without disbursing anything if the underlying request is cancelled, which is not communicated in the NatSpec within `ISynthetixDepositContract`.

RECOMMENDATION

Expand the NatSpec on `disburseWithdrawal` to document that that no disbursal is expected when the request has been cancelled and that `disburseWithdrawal` can finalize the status of the request.

RESOLUTION

Status: Acknowledged.

LOW **L-01**

ACKNOWLEDGED

Collateral Cannot Be Added In Deploy

CATEGORY	Logical Error
LOCATION	DeploySynthetixDepositContract.s.sol, PostDeployment.s.sol
REVIEW	Remediation Review 2

— DESCRIPTION

The deployment and post-deployment scripts schedule/execute `addCollateral` with only the token address, but the Deposit contract requires a `CollateralConfig` argument. This causes all scheduled collateral additions to revert at execution time.

— RECOMMENDATION

Update `addCollateralData` in `DeploySynthetixDepositContract` and `addData` in `PostDeployment` to supply a `CollateralConfig` rather than a token address.

— RESOLUTION

Status: Acknowledged.

LOW **L-02**

ACKNOWLEDGED

CowSwap Warnings

CATEGORY	Warning
LOCATION	Global
REVIEW	Remediation Review 2

— DESCRIPTION

According to spec, several situations will cause the system to have to sell non-USDT collaterals back into USDT, such as full trader liquidations. When accounting for use cases such as this, it is important to note that CowSwap is not atomic and may provide partial fills.

— RECOMMENDATION

Ensure the rest of the collateral exchange system is capable of supporting delayed/partial swaps.

— RESOLUTION

Status: Acknowledged.

LOW **L-03**

ACKNOWLEDGED

Token Withdrawal Paused After User Requested Withdraw

CATEGORY Warning
LOCATION SynthetixDepositContract.sol
REVIEW Remediation Review 2

DESCRIPTION

A user can submit a valid withdrawal request off-chain, which will then be relayed on-chain to request the withdrawal and later disburse it. If prior to disbursal the withdrawals were paused for a particular token, the entire batch would be DoS'd.

RECOMMENDATION

Ensure off-chain simulation architecture is resilient enough to adjust `_requestIds` accordingly.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-01**

ACKNOWLEDGED

Invalid MockToken Decimals Used For Testing

CATEGORY	Logical Error
LOCATION	DeployDevSetup.s.sol
REVIEW	Remediation Review 2

DESCRIPTION

`MockToken.mint` uses the `_decimals` parameter to scale the initial supply, but `decimals()` is overridden to always return 6. WETH is instantiated with 18 as `_decimals` and a WETH-oriented config in `1e18`, but `decimals()` still returns 6. This inconsistency will skew deposit/limit checks and can lead to confusing or incorrect test behavior.

RECOMMENDATION

Align `decimals()` with the intended token decimals.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-02

RESOLVED

Incorrect MIN_ENVELOPE_LENGTH Comment

CATEGORY Documentation
LOCATION SynthetixDepositContract.sol: 115
REVIEW Remediation Review 2

DESCRIPTION

The comment for the `MIN_ENVELOPE_LENGTH` is incorrect: `/// 12 head words (12*32) + bytes length (32) + padded 65-byte sig (96) = 384`

The calculation above (`12 * 32 + 32 + 96`) does not equal 384.

RECOMMENDATION

Update the comment to accurately reflect the `MIN_ENVELOPE_LENGTH` calculation.

RESOLUTION

Status: Resolved.

INFORMATIONAL **I-03**

ACKNOWLEDGED

Request ID Can Be Reused

CATEGORY	Warning
LOCATION	SynthetixDepositContract.sol
REVIEW	Remediation Review 2

DESCRIPTION

Request ID's can be reused in multiple privileged functions when they have already been finalized. For example, the Watcher can call `castWatcherVotes` on an already expired/validated request id as a no-op that will waste gas.

RECOMMENDATION

Ensure the off-chain infrastructure is not reusing finalized id's in the typical withdrawal lifecycle.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-04**

ACKNOWLEDGED

Parameters Can Be More Granular

CATEGORY Warning
LOCATION SynthetixDepositContract.sol
REVIEW Remediation Review 2

DESCRIPTION

Multiple parameters in the Deposit contract act globally for all tokens included `slippageToleranceBps`, `oracleStaleTimeout`, etc. It may be prudent to have more granular configurations per-token instead.

RECOMMENDATION

Consider having these parameters be part of a collateral's config.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-05**

ACKNOWLEDGED

Specific Swap Pause Can Be Added

CATEGORY	Warning
LOCATION	SynthetixDepositContract.sol
REVIEW	Remediation Review 2

DESCRIPTION

Considering that swaps make the `totalDeposited` and `userBalance` for a token out-of-line and are one of the ways funds leave the Deposit contract, it may be prudent to have a specific swap toggle without relying on disable an entire `sellToken`'s config.

RECOMMENDATION

Consider having a pause variable specifically used in `isValidSignature` to prevent swaps.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-06**

ACKNOWLEDGED

Unclear Error InvalidTradeParams

CATEGORY Warning
LOCATION SynthetixDepositContract.sol
REVIEW Remediation Review 2

DESCRIPTION

The error `InvalidTradeParams()` is raised for multiple reasons including but not limited to: slippage exceeded through swap, order buy/sell amount being 0, incorrect order `sellTokenBalance`, etc. It may be better to have specific reverts for specific issues to make for easier debugging.

RECOMMENDATION

Consider making reverts more specific.

RESOLUTION

Status: Acknowledged.

§ 05

Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

§ 06

About Guardian.

Guardian is a smart contract security firm. Every engagement is worked by two independent teams competing on the same scope, each with its own tooling and methods, so no single reviewer's blind spot survives.

\$45B+DIGITAL ASSETS UNDER
REVIEW**200+**

ENGAGEMENTS COMPLETED

500+CRITICAL & HIGH FINDINGS
REPORTED

Reach us at audits@guardianaudits.com.